

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине: «ОМО»
Тема: «Рекуррентные нейронные сети»

Выполнил:
Студент 3-го курса
Группы АС-66
Осовец А.О.
Проверил:
Крощенко А.А.

Брест 2025

Вариант 7

1. Выполнить моделирование прогнозирующей нелинейной ИНС. Для генерации обучающих и тестовых данных использовать функцию. Для прогнозирования использовать многослойную ИНС с одним скрытым слоем. В качестве функций активации для скрытого слоя использовать сигмоидную функцию, для выходного - линейную.

№ варианта	a	b	c	d	Кол-во входов ИНС	Кол-во НЭ в скрытом слое
1	0.1	0.1	0.05	0.1	6	2
2	0.2	0.2	0.06	0.2	8	3
3	0.3	0.3	0.07	0.3	10	4
4	0.4	0.4	0.08	0.4	6	2
5	0.1	0.5	0.09	0.5	8	3
6	0.2	0.6	0.05	0.6	10	4
7	0.3	0.1	0.06	0.1	6	2
8	0.4	0.2	0.07	0.2	8	3
9	0.1	0.3	0.08	0.3	10	4
10	0.2	0.4	0.09	0.4	6	2
11	0.3	0.5	0.05	0.5	8	3

```

import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import pandas as pd

# === 1. Генерация обучающих и тестовых данных ===
def target_function(x, a=0.3, b=0.1, c=0.06, d=0.1):
    return a * torch.cos(b * x) + c * torch.sin(d * x)

# Параметры варианта
a, b, c, d = 0.3, 0.1, 0.06, 0.1
input_size = 6
hidden_size = 2
num_samples = 200
train_ratio = 0.8

# Входы: 6 признаков — степени x
x = torch.linspace(0, 10, num_samples).reshape(-1, 1)
X = torch.cat([x ** i for i in range(1, input_size + 1)], dim=1)
y = target_function(x, a, b, c, d)

# Train/Test split
split_idx = int(num_samples * train_ratio)
X_train, X_test = X[:split_idx], X[split_idx:]
y_train, y_test = y[:split_idx], y[split_idx:]

print("Задание 3: Данные сгенерированы для функции  $y = a \cdot \cos(bx) + c \cdot \sin(dx)$ ")
print(f"Размер обучающей выборки: {X_train.shape}, тестовой: {X_test.shape}")

```

=== 2. Архитектура РНС Элмана ===

```
class ElmanRNN(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.hidden_dim = hidden_dim
        # RNN с tanh внутри (ограничение PyTorch)
        self.rnn = nn.RNN(input_dim, hidden_dim, nonlinearity="tanh", batch_first=True)
        self.sigmoid = nn.Sigmoid() # добавляем сигмоид вручную
        self.fc = nn.Linear(hidden_dim, 1) # линейный выход

    def forward(self, x):
        # РНС ожидает входы в формате [batch, seq, features]
        x = x.unsqueeze(1) # добавляем размерность seq=1
        out, _ = self.rnn(x)
        out = self.sigmoid(out[:, -1, :]) # сигмоид на скрытом слое
        out = self.fc(out) # линейный выход
        return out
```

```
model = ElmanRNN(input_size, hidden_size)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

=== 3. Обучение модели ===

```
losses = []
epochs = 500
```

```
for epoch in range(epochs):
    model.train()
    y_pred = model(X_train)
    loss = criterion(y_pred, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    losses.append(loss.item())
```

```
print("Обучение завершено. Минимальная ошибка:", min(losses))
```

=== 4. График ошибки по эпохам ===

```
plt.figure(figsize=(10, 4))
plt.plot(losses)
plt.title("График изменения ошибки по эпохам (РНС Элмана)")
plt.xlabel("Эпоха")
plt.ylabel("MSE")
plt.grid(True)
plt.show()
```

=== 5. График прогнозируемой функции на обучающем участке ===

```
model.eval()
with torch.no_grad():
    y_train_pred = model(X_train)

plt.figure(figsize=(10, 4))
plt.plot(x[:split_idx], y_train, label="Эталон")
plt.plot(x[:split_idx], y_train_pred, label="Прогноз РНС Элмана")
plt.title("Прогнозируемая функция на обучающем участке")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()
```

=== 6. Таблица результатов обучения ===

```
train_table = pd.DataFrame({
    "Эталонное значение": y_train.squeeze().numpy(),
    "Полученное значение": y_train_pred.squeeze().numpy(),
})
```

```

        "Отклонение": (y_train_pred - y_train).squeeze().numpy()
    })
    print("\nРезультаты обучения (первые строки):")
    print(train_table.head())

# === 7. Таблица результатов прогнозирования ===
with torch.no_grad():
    y_test_pred = model(X_test)
    test_table = pd.DataFrame({
        "Эталонное значение": y_test.squeeze().numpy(),
        "Полученное значение": y_test_pred.squeeze().numpy(),
        "Отклонение": (y_test_pred - y_test).squeeze().numpy()
    })
    print("\nРезультаты прогнозирования (первые строки):")
    print(test_table.head())

```

Результат:

Задание 3: Данные сгенерированы для функции $y = a \cdot \cos(bx) + c \cdot \sin(dx)$

Размер обучающей выборки: `torch.Size([160, 6])`, тестовой: `torch.Size([40, 6])`

Обучение завершено. Минимальная ошибка: 0.0001011862768791616

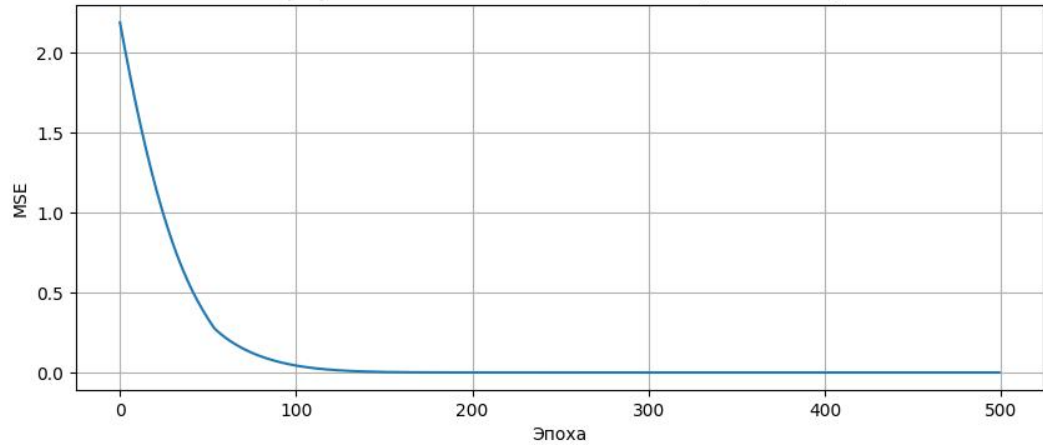
Результаты обучения (первые строки):

	Эталонное значение	Полученное значение	Отклонение
0	0.300000	0.286533	-0.013467
1	0.300298	0.286178	-0.014119
2	0.300588	0.285850	-0.014738
3	0.300870	0.285547	-0.015323
4	0.301145	0.285273	-0.015873

Результаты прогнозирования (первые строки):

	Эталонное значение	Полученное значение	Отклонение
0	0.251354	0.289812	0.038458
1	0.250475	0.289812	0.039337
2	0.249589	0.289812	0.040223
3	0.248696	0.289812	0.041116
4	0.247798	0.289812	0.042014

График изменения ошибки по эпохам (РНС Элмана)



Прогнозируемая функция на обучающем участке

